

Binary File Protocol v1

Author: Antara Utane

This specification defines the binary messages exchanged by bserve and bcurl. The course requires TCP, a fixed frame header, ten numbered headers with a literal fallback, error handling, persistence, unknown-frame skipping, and hexdumps. The exact formats below are this project's design. **MUST** means required for compatible v1 endpoints.

1. Transport and frame header

Use **TCP**. All integers are unsigned; multibyte integers use **big-endian** byte order. u8, u16, and u32 mean one, two, and four bytes. Lengths count bytes, not characters. Reads may split or combine frames: receive exactly eight header bytes, then exactly the declared payload.

Byte offset	Width	Field	Meaning and allowed values
0-3	32 bits	Payload length L	Bytes after the header; 0-16,777,216
4	8 bits	Type	1 = REQUEST; 2 = RESPONSE; others unknown
5	8 bits	Version	MUST be 1
6-7	16 bits	Reserved	MUST be 0

Total frame size is **8 + L**. A 32-bit length is easy to encode and leaves expansion room; the **16 MiB payload cap** bounds memory use. Eight bits suffice for type/version; two reserved bytes complete the eight-byte header and leave room for future flags. Unlike HTTP/2's 24/8/8/31 arrangement, sequential requests need no stream ID. Only REQUEST and RESPONSE are known types; errors use RESPONSE.

2. Payload layouts

Fields follow one another, with no padding or terminators.

REQUEST (type 1, client -> server):

method:u8 | path_length:u16 | path bytes | header_count:u16 | headers

Method **MUST** be 1 (GET). Path length is 1-65,535; path bytes are UTF-8 text. Header count is 0-65,535; parse exactly that many entries. There is no request body and no extra payload data is permitted.

RESPONSE (type 2, server -> client):

status:u16 | header_count:u16 | headers | body bytes

Status is a numeric code from 100 to 599, not text. The server uses **200** for success, **400** for malformed requests, **404** for missing files or paths that are not regular files, and **500** for unexpected file errors or a response exceeding the cap. Each response is final. Header count is 0-65,535.

After parsing the headers, **all remaining payload bytes are the body**. Body length is L minus the bytes used by status, count, and header entries. There is no separate body-length field. Return exact file bytes, including empty and binary files. Error bodies are short UTF-8 text; wording is not prescribed.

3. Header encoding

Numbered entry: name_id:u8 | value_length:u16 | value bytes.

Literal entry: 0:u8 | name_length:u16 | name bytes | value_length:u16 | value bytes.

IDs 1-10 name dictionary entries; 11-255 are malformed. Literal names contain only lowercase ASCII letters, digits, and hyphens; name length is 1-65,535. Values are UTF-8 text, possibly empty; value length is 0-65,535. All fields **MUST** fit the payload. Dictionary names may also use literal form. Receivers **MUST** accept unfamiliar literal names. Order and repeated names have no special meaning.

ID	Name	Normal sender and value
1	host	Client: requested host and port
2	user-agent	Client: bcurl/1
3	accept	Client: */*
4	accept-encoding	Client: identity
5	connection	Client: keep-alive
6	content-type	Server: file type or application/octet-stream; errors text/plain
7	content-length	Server: decimal body byte count
8	server	Server: bserve/1
9	cache-control	Server: no-store
10	x-protocol-version	Server: 1

Normal clients send IDs 1-5; normal servers send IDs 6-10, including errors. Receivers also accept other correctly encoded lists. Headers describe the message but do not change how it is read. Framing determines body length; content-length never overrides it. The other header values do not change protocol behavior.

4. Paths, malformed input, and persistence

Paths MUST start with /; / means /index.html. Otherwise split on /, ignore empty and . segments, and join under the supplied root. Reject . . segments, NUL, backslash, colon, or a target whose resolved location is outside the root.

Do not percent-decode or interpret queries/fragments: characters are literal filename characters. Invalid/unsafe paths return 400; missing files and directories return 404. Every resolved target MUST stay within the root.

Malformed input includes invalid version/reserved fields, a known type in the wrong direction, unsupported method, invalid text/path/header ID, missing fields, fields extending beyond the payload, or extra REQUEST bytes. For a complete eight-byte header with an in-limit L, the server MUST consume the payload, return one 400, and keep the connection open. Do not scan for a guessed boundary.

For an oversized L, return 400 if possible, then close without reading the payload. If EOF occurs partway through a frame, return 400 if the socket permits, then close; a broken connection may prevent a reply. Clean EOF between frames ends normally. Closing for oversized or incomplete frames is a design choice; complete malformed requests keep the connection open. A client receiving malformed input closes and exits unsuccessfully without sending an error frame.

Allow one outstanding request. The server loops: **read frame -> handle/skip -> reply when required -> read next frame**. It MUST keep the connection open after 200, 404, 500, and recoverable 400. The client makes one connection attempt, sends one request, reads its response, then closes. It MUST NOT reconnect or retry.

5. Unknown frames, tools, and evidence

A receiver that encounters an unknown frame type MUST skip it cleanly using the frame length. For a valid envelope, discard exactly L bytes in small chunks, generate no response solely for that frame, and continue reading. Unknown zero-length frames are valid. Envelope errors/incomplete payloads follow the rules above. A waiting client continues past unknown frames until its response arrives.

Assignment commands: ./bserve ./www 9000 and ./bcurl -v localhost:9000/index.html. Windows equivalents use python bserve and python bcurl with those arguments. Client input is host:port/path, with explicit port 1-65,535; localhost suffices for demonstrations.

Body bytes go unchanged to binary stdout. Diagnostics and -v hexdumps go to stderr. Verbose mode dumps every sent/received frame, including skipped frames, with direction, byte offsets, and hex bytes; incomplete captures are labelled partial. Exit 0 for 2xx and 1 for every failure, including 4xx/5xx, invalid input, malformed response, and connection errors.

The course submission includes the program, this two-page specification, and an **annotated hexdump of one actual complete request/response**, explaining every field and byte range.